

AI-Assisted Development — Exam Submission

HotFixAmbulance — AI-Powered Production-Error Triage for .NET Web APIs

Author: Stanislav Stanev

AI tool used: Claude Code — Anthropic’s agentic CLI — running the **Claude Opus 4.8** model (1M-context)

Repository: <https://github.com/myPOSTech/mps-banking-hot-fix-ambulance>

Working-system evidence: <docs/evidence/working-system-evidence.pdf>

1. Project Overview

The idea

On-call engineers waste the first, most expensive minutes of an incident doing the same manual ritual: open Kibana, filter the last few hours of errors, eyeball the stack traces, guess the root cause, and go hunting through Git for the offending change. **HotFixAmbulance** automates that ritual. It is a triage tool that, for a chosen .NET Web API and time window, pulls the recent error logs, groups them into distinct problems, and — for each problem — uses a **local Qwen LLM** to write two human-readable columns: “**Suggestion for Error**” (what the error means) and “**How to fix**” (a concrete remediation), grounded in the service’s own Git history. The result is a ranked, explained, actionable error table in a web UI, plus a CLI for terminal/CI use.

System requirements

Functional

- Fetch Fatal/Error/Warning logs for a named API over a relative (last N h) or absolute time window from **Elasticsearch**.
- Group logs by exception fingerprint (exception type + normalized message + endpoint) and rank by severity, count, recency.
- For each group, produce an AI **Suggestion** and **How-to-fix**, using a **locally hosted Qwen model** and grounding the answer in Git blame + related commits.
- Persist every run; expose it through a REST API and a paginated React table; provide a CLI that triages and prints JSON/table output.
- The UI must make it **visible and verifiable** that a group was analysed by the LLM (a per-group 🤖 **Qwen** badge).

Non-functional

- **Graceful degradation:** the LLM is optional — if the model is unreachable, the system silently falls back to a deterministic Git-history heuristic and never fails a run.
- **Local & private:** the model runs on the developer’s machine in Docker (CPU-only), no data leaves the host.
- **Reproducible:** one command (`scripts/demo.ps1`) brings up the whole stack (Qwen, Elasticsearch, SQL Server), generates sample errors, runs triage, and opens the UI.

Tech stack

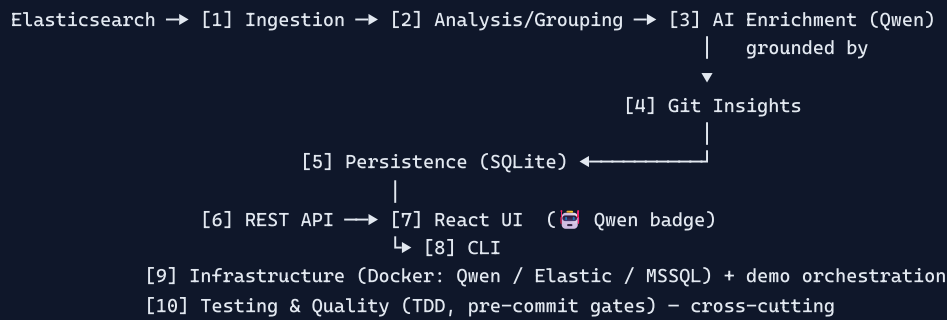
.NET 9 / C# 13 (backend, CLI) · ASP.NET Core Minimal APIs · Entity Framework Core + SQLite · Elasticsearch 8 ·

LibGit2Sharp · React 18 + TypeScript + Vite + TanStack Table + Tailwind · Ollama-runtime serving **Qwen 2.5 (3B)** ·

Docker Compose · PowerShell automation · xUnit + FluentAssertions + NSubstitute (backend) · Vitest + Testing Library (frontend).

2. System Architecture — Modules

The system was decomposed (with AI assistance) into focused technological modules, each a separate .NET project or front-end/infra unit with a single responsibility and a clear seam (interface) to its neighbours.



3. Development Process per Module

3.1 Log Ingestion (Elasticsearch)

3.2 Error Analysis & Grouping

Testing strategy. Pure unit tests assert grouping keys, ranking order and correlation-id counting — fast and exhaustive because the layer is side-effect-free.

AI tool choice. Claude Code — strong at generating table-driven ranking tests and edge cases.

Prompts: “Group logs by a stable fingerprint and rank Fatal>Error>Warning, then count desc, then last-seen desc.” · “Add a normalizer that replaces GUIDs/numbers so the same error collapses into one group.”

3.3 AI Enrichment with Qwen (the centrepiece)

Approach & reasoning. The AI columns are owned by an `IGroupEnricher` seam so the strategy is swappable from config (`Analysis:Strategy`). `LlmGroupEnricher` builds a strict-JSON prompt (`{suggestion, howToFix}`) from the group + Git evidence, calls the model via `ILlmClient` (`OllamaLlmClient` , `temperature 0.2` , `format:"json"`), and on **any** failure (timeout, unreachable, bad JSON) degrades to the deterministic Git heuristic. A successful call tags the group `AnalyzedBy = "Llm"` ; that single fact drives the UI badge.

Step-by-step workflow. (1) `LlmPromptBuilder` (system contract + fact dump); (2) `OllamaLlmClient` that never throws (returns `null` on failure); (3) `LlmGroupEnricher` with fallback; (4) propagate per-group `AnalyzedBy` through `TriageService` → `ErrorGroup` → API → UI.

Testing strategy. Unit tests with a substituted `ILlmClient` assert: valid JSON → `Llm` ; null/garbage → heuristic fallback; and that `TriageService` stamps each group with the enricher’s source.

AI tool choice. Claude Code — it implemented the fallback contract, the run-level `Mixed/Llm/Heuristic` aggregation, and the tests in one TDD loop.

Prompts: “Add an LLM enricher that asks the model for {suggestion, howToFix} as JSON and falls back to the Git heuristic on any error — never throw.” · “Make sure every group records which strategy produced it, and surface a per-group marker in the UI proving Qwen was used.”

3.4 Git-History Grounding

Approach & reasoning. An LLM answer is only useful if grounded in the real code, so `FixHintBuilder` (behind `IFixHintSource`) uses `LibGit2Sharp` to fetch the offending file’s blame + a code snippet and search `origin/main` for related commits; this evidence is fed to both the heuristic and the Qwen prompt.

Step-by-step workflow. (1) `IGitRepoCache` to clone/pull per API; (2) `IGitHistoryReader` for blame + keyword commit search; (3) `FixHintBuilder` to assemble a compact evidence string.

Testing strategy. Unit tests substitute the repo cache/reader and assert the formatted hint contains the expected `file:line` , blame and commit SHAs.

AI tool choice. Claude Code — handled the `LibGit2Sharp` specifics and kept the evidence format testable.

Prompts: “Given a stack file+line, return blame + a code snippet from origin/main and the related commits.”

3.5 Persistence & REST API

Approach & reasoning. Runs are stored as a row with the groups serialised to JSON (SQLite), keeping the schema trivial and migration-free; the API is ASP.NET Core Minimal APIs with server-side paging/sorting (`GroupPager`).

Step-by-step workflow. (1) `HotFixDbContext` + `TriageRun` (incl. nullable `AnalyzedBy` for back-compat); (2) repository; (3) endpoints: run triage, latest, by-id, paged groups, history.

Testing strategy. Integration tests with `WebApplicationFactory` hit the real endpoints (in-memory/SQLite), asserting status codes, paging bounds and the `analyzedBy` field round-trips.

AI tool choice. Claude Code — generated endpoints + integration tests and kept DTOs in sync with the frontend types.

Prompts: “Expose paginated groups with sort/dir validation and an allowed page-size list, plus a run-by-id endpoint.”

3.6 Frontend UI + the Qwen badge

Approach & reasoning. A `TanStack-Table` grid with toggleable columns; the AI columns render a small violet

 Qwen badge iff `analyzedBy == 'Llm'` — the visible proof the model was used.

Step-by-step workflow. (1) typed API client + `ErrorGroup` type incl. `analyzedBy`; (2) `QwenBadge` component; (3) render it in the Suggestion and How-to-fix cells; (4) metrics panel (“AI Insights Generated”).

Testing strategy. Vitest + Testing Library: badge appears for `Llm`, is absent for `Heuristic` / `null` — written **before** the component (TDD red→green).

AI tool choice. Claude Code — wrote the failing tests first, then the component, matching the repo’s TDD rule.

Prompts: “Render a Qwen badge on the AI columns only when analyzedBy is 'Llm'; add Vitest tests for all three cases first.”

3.7 Infrastructure & DevOps

Approach & reasoning. Mirror the existing `infra/<service>` pattern: a CPU-only Qwen runtime in Docker Compose (`infra/qwen`), an idempotent `bootstrap.ps1` that pre-pulls `qwen2.5:3b`, and a `demo.ps1` that orchestrates the whole stack and **asserts** the produced run was analysed by the LLM before opening the UI.

Step-by-step workflow. (1) compose + healthcheck; (2) bootstrap (pull + chat probe); (3) corporate-CA injection (`export-host-ca.ps1` + `docker-compose.corp.yml`) for TLS-intercepting proxies; (4) wire `-WithLlm` /default LLM env into `demo.ps1`.

Testing strategy. Each script is verified live: compose `config`, container health, `/api/tags`, model present, idempotent re-run, and an end-to-end demo asserting `analyzedBy ∈ {Llm, Mixed}`.

AI tool choice. Claude Code — it ran the Docker/PowerShell commands, diagnosed failures from their output, and iterated.

Prompts: “Make Qwen part of the infrastructure in Docker like Elasticsearch, and guarantee the model is present before the app starts.” · “The container can’t verify TLS through our proxy — inject the corporate CA.”

3.8 Testing & Quality (cross-cutting)

Approach & reasoning. A mandatory TDD cycle (failing test first) and a **pre-commit hook** that blocks any commit unless `dotnet test`, `dotnet format`, frontend tests and lint all pass — so `main` /integration stays green.

Step-by-step workflow. Write the failing test → minimal implementation → run → commit (hook gates it).

Testing strategy. 160 backend unit tests + 18 integration tests + 34 frontend tests, all green at each commit.

AI tool choice. Claude Code — drove the red→green→commit loop and respected the project’s skills/hooks.

Prompts: “Add the failing test for AnalyzedBy first, watch it fail, then implement and run the suite.”

4. Challenges & Tool Comparison

Biggest challenges

- **Corporate TLS interception.** The Docker container could not `ollama pull` the model — `x509: certificate signed by unknown authority` — because the corporate proxy MITM-terminates TLS and the container lacked the corporate root CA. **Solved** by exporting the host CA bundle (`export-host-ca.ps1`) and mounting it via a `docker-compose.corp.yml` overlay with `SSL_CERT_FILE`. This was the single hardest, most environment-specific problem.
- **A latent DI bug surfaced by the new feature.** The CLI had never been wired to the `IGroupEnricher` seam added in a prior phase, so it crashed with “Unable to resolve service for type IGroupEnricher.” Found it by running the CLI; fixed by registering `AddHotFixGroupEnrichment` — which also enabled the CLI to use Qwen.
- **CPU inference latency.** One LLM call per group, sequential, on CPU (~10 s each) makes a 38-group run take minutes; mitigated by warming the model and raising the API client timeout. A real trade-off of running a private model locally.
- **Tooling papercuts.** PowerShell 5.1 mis-parsing a UTF-8 script because of em-dash characters (replaced with ASCII); Chrome headless needing absolute paths and an origin before `localStorage`; SQLite file-locks when a running API blocked the build. Each was diagnosed from command output and fixed.

Which tool helped the most, and why

Claude Code running the **Claude Opus 4.8** model (1M-context) was the primary and most valuable tool. Beyond code generation it could **run** the project — execute

Docker/PowerShell/dotnet, read the failures, and iterate — which is what actually solved the infrastructure problems (the CA injection, the DI crash, the parse bug). Its skills/sub-agents kept the work disciplined (plan → TDD → verify → commit with a passing pre-commit gate), and it could reason across the whole repo (backend + frontend + infra) to keep types, DTOs and the demo orchestration consistent end-to-end.

What I would improve next

- **Batch / parallelise** the per-group LLM calls (or stream) to cut run time; optionally cache by fingerprint.
- Add a **GPU compose overlay** for much faster inference where a GPU is available.
- Make the persisted strategy tag itself read **Qwen** (currently the internal tag stays the generic **Llm**, while the UI shows “Qwen”).
- Share one SQLite store between CLI and API so a CLI-produced run is viewable in the UI without a second pass.
- Add a small **eval harness** that scores the model’s suggestions against known root causes.

5. Working System Evidence

See [docs/evidence/working-system-evidence.pdf](#) (4 captioned figures). Highlights:

- **Figure 1 — Overview:** a real run for **demo-api** — **73 logs** → **38 groups**, with **38 “AI Insights Generated”** and **38 “Fix Recommendations.”**
- **Figure 2 — Proof of Qwen:** every AI cell shows the  **Qwen** badge and genuine, context-specific model text

(e.g. “...failed database save operation at line 311 in DemoDatabase.cs...”, “...invalid SQL object name 'Pricing.Records'...”).

- **Figure 3 — Full table:** all **38 groups** carry the badge (run-level **analyzedBy = "Llm"**).
- **Figure 4 — Terminal:** **docker ps** (Qwen healthy on :11434), **ollama list** (**qwen2.5:3b** , 1.9 GB), API run **analyzedBy = Llm** with **withSuggestions = 38** / **withFixes = 38** , and the container at **1177 % CPU** during inference.

Reproduce locally:

```
powershell -File scripts/demo.ps1 -WithElastic -KeepRunning
# then open the printed UI URL – every analysed group shows the  Qwen badge
```

6. Repository

GitHub: <https://github.com/myPOSTech/mps-banking-hot-fix-ambulance>

(Feature work on branch **integration-llm**.)

7. Author’s Notes

This is **my own idea project**. Beyond the exam, I intend to present it to **myPOS management** as a **working prototype** for everyday use by the **.NET teams** — a practical, privacy-respecting way to shorten incident triage in a fintech environment where production data must stay in-house.

Why it is worth adopting:

- **Faster incident response (lower MTTR).** On-call engineers open a ranked, *explained* error list instead of scrolling raw Kibana logs — the “what” and the “how to fix” are already written for each distinct problem.
- **Lower skill barrier.** Junior engineers get plain-English root-cause hints and remediation steps ****grounded in the team’s own Git history**** (blame + related commits), not generic advice.

- **Consistent, repeatable triage.** Every incident is analysed the same way, with the same evidence, so handovers and post-mortems start from a common baseline.
- **Privacy & compliance by design.** The model runs **locally in Docker (CPU-only)** — no logs, stack traces or source context leave myPOS infrastructure. Critical for banking/PCI contexts where cloud LLMs are a non-starter.
- **No per-token cost.** Uses a small open model ([qwen2.5:3b](#)) locally; there are no cloud-API usage fees.
- **Fits the existing stack.** .NET, Serilog, Elasticsearch and Git are tools the teams already run; adoption is additive, not a migration.
- **Safe to roll out.** The LLM is optional and **degrades gracefully** — if the model is down, triage still works via the deterministic Git heuristic, so it can be enabled per-team without risk.
- **Extensible.** Pluggable analysis strategy (heuristic ↔ LLM), pluggable model and per-API configuration mean it can grow from a prototype to a shared internal service.

Proposed next step at myPOS: pilot it with one .NET team on a non-critical service for two weeks, measure the change in triage time and engineer feedback, then decide on a shared, GPU-backed internal deployment.

8. How to Reproduce the Full End-to-End Test

This section lets a reviewer reproduce the **entire experimental setup** shown in §5 from a clean checkout. One PowerShell command brings up the whole stack (Qwen + Elasticsearch + SQL Server), generates sample errors, runs the triage on Qwen, starts the API + UI, asserts the run was analysed by the LLM, and opens the browser on it.

8.1 Prerequisites

- **Windows 10/11 + PowerShell 5.1** (the automation uses Windows/PowerShell features).
- **Docker Desktop** installed and **running** (Linux containers).
- **.NET 10 SDK, Node.js 18+ & npm, Git.**
- **Disk/network:** the first run downloads ~**6 GB** of images + models (Ollama image, [qwen2.5:3b](#) ≈ 2 GB,

Elasticsearch, SQL Server). Subsequent runs reuse the cached volumes and are much faster.

- A free [11434](#) (Qwen), [5283](#) (API), [5173](#) (frontend), [9200](#) (Elasticsearch), [14333](#) (SQL Server).

8.2 One-time setup

```
# 1) Clone and switch to the feature branch
git clone https://github.com/myPOSTech/mps-banking-hot-fix-ambulance.git
cd mps-banking-hot-fix-ambulance
git checkout integration-llm

# 2) Restore tooling and install the pre-commit hook (.NET restore + npm install)
powershell -File scripts/bootstrap.ps1

# 3) Make sure Docker Desktop is running
docker version
```

8.3 Run the full end-to-end demo

```
powershell -File scripts/demo.ps1 -WithElastic -KeepRunning
```

This single command performs, in order:

1. **Starts the Qwen runtime** ([infra/qwen](#) , Docker, CPU-only) and runs its bootstrap, which **pulls** [qwen2.5:3b](#) if absent and verifies it with a JSON [/api/chat](#) probe. (First run downloads ~2 GB — be patient.)
2. Sets [Analysis:Strategy=Llm](#) + the [Llm](#) endpoint/model for every child process.

3. **Starts SQL Server and Elasticsearch** (Docker) and waits for health.
4. **Builds and starts** `demo-api`, hammers its endpoints to **generate realistic error traffic**, then reshapes the logs into Elasticsearch via the ingest pipeline.
5. **Runs the CLI triage** on Qwen, then **builds and starts the API** (`:5283`) and the **frontend** (`:5173`).
6. **Creates a triage run via the API** and **asserts** the result's `analyzedBy` is `Llm / Mixed` — i.e. it **fails loudly if Qwen was not actually used**.
7. **Opens your default browser** on the produced run (see §8.5 for the link and what is shown) — every analysed group shows the  **Qwen** badge.

8.4 Expected console output (key markers)

```
[demo] LLM strategy enabled: provider=Qwen model=qwen2.5:3b endpoint=http://localhost:11434
[qwen-bootstrap] chat probe: ok
[demo] Producing error traffic
[es-bootstrap] Reindex: total=94 created=94 updated=0 failures=0
[demo] Running CLI: hot-fix-ambulance demo-api --lookback 1h
[demo] Starting HotFixAmbulance.Api on http://localhost:5283
[demo] Creating the run via API: POST /api/triage/demo-api?lookbackHours=1
[demo] API run id=<guid> analyzedBy=Llm groups=38
[demo] Verified: this run was analyzed by Qwen (analyzedBy=Llm). The UI shows the Qwen badge.
[demo] Opening UI: http://localhost:5173/?analysisId=<guid>&api=demo-api
```

8.5 Open the result in the browser

When the demo finishes it **automatically opens your default browser** on the produced run. The URL looks like:

```
http://localhost:5173/?analysisId=<run-guid>&api=demo-api
```

The exact `<run-guid>` is printed in the console (see §8.4 — the `Opening UI:` line). While the API (`:5283`) and the frontend (`:5173`) stay running you can also open the result manually at any time:

- **Latest run, no id needed:** open `http://localhost:5173/?api=demo-api` — the UI loads the most recent run for `demo-api`.
- **A specific run:** paste the full URL using the `analysisId` printed in the console.

What you will see (Figure 5). The triage dashboard for `demo-api`: a header with the run summary

(**73 log(s) in 38 group(s)**), a metrics strip (**Total Errors Detected**, **AI Insights Generated**,

Fix Recommendations), and the ranked error table. In the “**Suggestion for Error**” and “**How to fix**”

columns, **every analysed group shows a violet**  **Qwen badge** above the model-written text — the visible proof

that the Qwen LLM produced the analysis. The **Column Settings** button (top-right) toggles which columns are

shown; the **Run analysis** button (top-right) re-runs the triage for the selected API and time range.

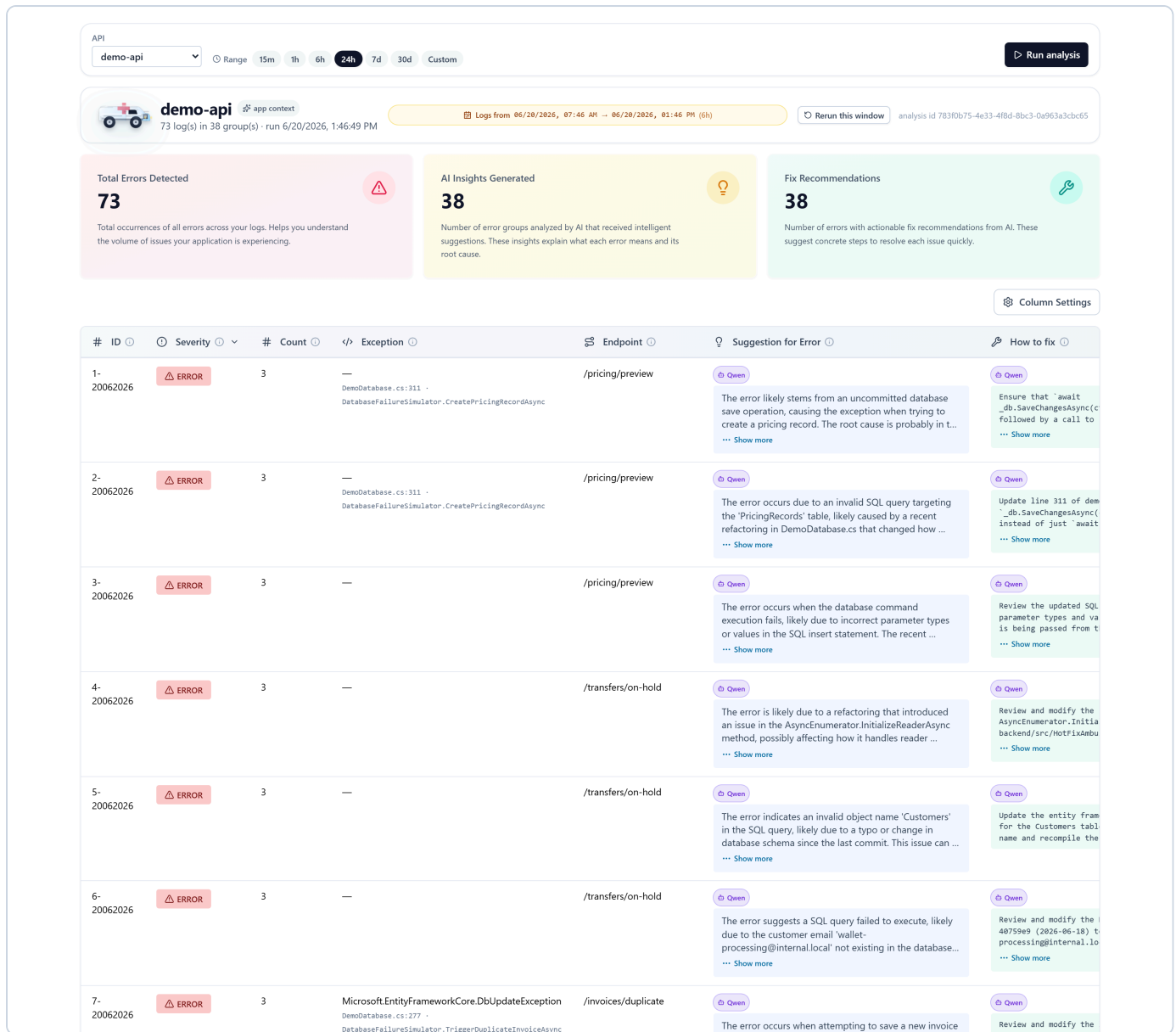


Figure 5 — The browser opened on the produced run: the demo-api triage dashboard. Every group's "Suggestion for Error" and "How to fix" cell shows a 🤖 Qwen badge above the model-generated text, proving the analysis came from the Qwen LLM.

8.6 Verify the LLM directly (optional, independent proof)

```
# The Qwen model is loaded in the container
docker exec hfa-qwen ollama list          # → qwen2.5:3b ... 1.9 GB

# Produce a run via the API and confirm it was analysed by the LLM
curl.exe -s -X POST "http://localhost:5283/api/triage/demo-api?lookbackHours=1"
# → JSON whose "analyzedBy" field is "Llm"

# Watch the Qwen container saturate the CPU during inference
docker stats hfa-qwen --no-stream        # → CPU ~1000%+ while a run is in flight
```

8.7 Troubleshooting

- **x509: certificate signed by unknown authority on model pull (corporate networks).** `demo.ps1` already handles this — it exports the host CA bundle (`infra/qwen/export-host-ca.ps1`) and starts the runtime with `infra/qwen/docker-compose.corp.yml` so the container trusts an intercepting proxy. On a normal home network no action is needed.
- **docker compose up failed** . Docker Desktop is not running — start it and re-run.

- **A port is busy.** Stop whatever holds `5283` / `5173` / `11434` , or close a previous demo run.
- **Model pull is slow.** It is a one-time ~2 GB download; the demo waits for it. Re-runs skip it.
- **Run without the LLM** (heuristic only, no Docker model): `powershell -File scripts/demo.ps1 -WithElastic -KeepRunning -SkipLlm` .

8.8 Teardown

```
docker compose -f infra/qwen/docker-compose.yml down          # keeps the model volume
docker compose -f infra/elasticsearch/docker-compose.yml down
docker compose -f infra/mssql/docker-compose.yml down
# add '-v' to any of the above to also delete its data/model volume
```

9. Project Timeline (from the Git history)

The project was built in **69 commits over four days (16–20 June 2026)**. Every commit was gated by a pre-commit hook that runs the full backend + frontend test suites, formatting and lint — `--no-verify` was never used — so each row below is a green, independently-inspectable checkpoint. The history is generalised by day, with the milestone reached and a few representative commit ids.

Day	Milestone	Key commits	What was done
16 Jun	End-to-end skeleton working	<code>981a210</code> , <code>bb62f32</code> , <code>0cd65da</code> , <code>b213806</code> , <code>2b061dd</code> , <code>bc7b613</code> , <code>b773ed7</code>	Scaffolded the repo + Claude tooling, then built the backend solution and Core domain (Severity / LogEntry / ErrorGroup, 16 tests) and every module in turn — Elastic ingestion, heuristic grouping, Git-insights (LibGit2Sharp), persistence + Minimal API, and the CLI — plus the React triage table, the demo-api with Serilog, and a Dockerized Elasticsearch stack. First full pipeline: ingest → analyse → git evidence → API / UI / CLI.
18 Jun	Realistic demo data + time windows	<code>acd1c28</code> , <code>40759e9</code> , <code>0a3b622</code> , <code>66a932e</code> , <code>d54056f</code>	Rewrote demo-api for real EF Core CRUD against Dockerized SQL Server (genuine DB failures: unique-constraint, timeouts); surfaced the offending source line + git blame in “How to fix”; added the <code>TimeWindow</code> value object (relative/absolute ranges), the frontend TimeRangePicker + API dropdown, and expandable cells.
19 Jun	Server-side pagination + LLM groundwork	<code>c5dd730</code> , <code>13e7027</code> , <code>b923f5c</code> , <code>e5670e9</code> , <code>d0111a6</code>	Shipped server-side pagination (PagedResult / TriageSummary / GroupPager, paged groups endpoint, TriageRunHeader, numbered footer); fixed fractional lookback hours; scaffolded the <code>HotFixAmbulance.Llm</code> project and the JSON-contract prompt builder.
20 Jun	Whole flow on local Qwen + visible proof + docs	<code>3831bc8</code> , <code>9d7f17e</code> , <code>c15520b</code> , <code>979252b</code> , <code>7e25e9a</code> , <code>3657a19</code>	Completed the LLM layer (ILlmClient / OllamaLlmClient, the IGroupEnricher seam, LlmGroupEnricher with graceful fallback, the Analysis:Strategy toggle); added the per-group <code>AnalyzedBy</code> marker and the 🚑 badge, then rebranded it to Qwen ; fixed a latent CLI DI break; Dockerised the Qwen runtime with model bootstrap + corporate-CA injection; ran the whole demo on Qwen and proved it via the UI badge; and produced the full exam documentation (write-up, evidence, merged PDF).

Reviewer note. The history reads as a disciplined, test-first progression: a working vertical slice first (16 Jun), then realism and UX (18 Jun), then scale/pagination and LLM groundwork (19 Jun), and finally the AI feature delivered end-to-end with proof and documentation (20 Jun). Each phase is a small, independently-tested commit, so any single step can be reviewed in isolation, and the green pre-commit gate on every commit means the `main` /integration branch was never left broken.